

탐구 - 증상 자연어



탐구

자연어를 통해 증상을 이용하면 어떤 질병이 의심되는지 간의로 판별해주는 프로그램 개발

탐구 동기

주말 혹은 보건 선생님께서 학교에 계시지 않을 때 아픈 증상 혹은 상처를 자연어로 말하여 간단한 질병의 여부 혹은 위험도 등을 미리 알 수 있는 프로그램이 있으면 하여 제작하게 되었다.

gpt에서 이전에 지원하였던 건강 처방 기능에서 증상에 대한 신뢰도를 파악하기 어렵다는 점을 참고하여 top-k 방식으로 증상에 따른 질병 확률이 높은 순으로 출력하고 증상에 대해 신뢰도가 떨어진다면 더 자세한 증상을 요구하도록 설계하는 것을 목표로 하였다. 실제 gpt,gemini,claude와 같은 자연어 처리 모델 자체를 구현하는 것은 힘들기에 데이터 분석,인코딩,전처리 방식을 통해 해결하기로 하였다.

데이터 수집

우선 가장 쉬운 데이터셋 부터 시도해보기로 하였다.

```
1  import kagglehub
2
3
4  path = kagglehub.dataset_download("kaushil268/disease-prediction-using-machine-learning")
5
6  print("Path to dataset files:", path)
```

학습&테스트 코드

```
1  import pandas as pd
2  import torch
3  import torch.nn as nn
4  from sklearn.preprocessing import LabelEncoder
5
6  # 데이터 로드
7  train = pd.read_csv("dataset-e/1/Training.csv")
8  test = pd.read_csv("dataset-e/1/Testing.csv")
9
10 for df in (train, test):
11     if "Unnamed: 133" in df.columns:
12         df.drop(columns=["Unnamed: 133"], inplace=True)
13
14 # train, test x, y 만들기
15 le = LabelEncoder()
16 X_train = torch.tensor(train.drop(columns=["prognosis"]).values, dtype=torch.float32)
17 y_train = torch.tensor(le.fit_transform(train["prognosis"]), dtype=torch.long)
18 X_test = torch.tensor(test.drop(columns=["prognosis"]).values, dtype=torch.float32)
19 y_test = torch.tensor(le.transform(test["prognosis"]), dtype=torch.long)
20
21 n_symptoms = X_train.shape[1]    # 132
22 n_diseases = len(le.classes_)    # 42
23
24 # 모델
25 model = nn.Sequential(
26     nn.Linear(n_symptoms, 64),
27     nn.ReLU(),
28     nn.Linear(64, n_diseases),
```

```
29 )
30
31
32 loss_fn = nn.CrossEntropyLoss()
33 optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
34
35 # 학습
36 for epoch in range(1000):
37     optimizer.zero_grad()
38     out = model(X_train)
39     loss = loss_fn(out, y_train)
40     loss.backward()
41     optimizer.step()
42     if (epoch + 1) % 20 == 0:
43         print(f"epoch {epoch+1}, loss {loss.item():.4f}")
44
45 # 추론 및 평가
46 model.eval()
47 with torch.no_grad():
48     pred = model(X_test).argmax(dim=1)
49     acc = (pred == y_test).float().mean()
50     print(f"\ntest: {acc.item():.4f}")
```

epoch	100	200	1000
테스트 정확도	0.9762	1.0000	1.0000

사실상 epoch가 100번을 넘어가면 예측 정확도가 100%를 넘었으며 이것은 데이터 자체가 너무 단순하며 테스트 데이터 자체에도 어떻게 보면 편향(너무 간단한 것이기도 함)이 있다고 볼 수 있다.
신경망은 단일 ReLU MLP로 구성

계획

- 임베딩 모델 만들기
- 데이터셋 가공 및 합체
- 학습 후 평가

그래서 기존 데이터를 분석하는 방식 보다는 데이터셋들을 여러 개 가지고 와서
증상들을 멀티 핫 벡터로 가공한 뒤에 문장으로 물었을 때 증상들만 인코딩을 통해 단어로 가져온 후 이에 맞는 질병이 무엇인지 분류하기로 하였다.

임베딩 모델

가장 큰 고비는 입력 문장을 정해진 증상들로 변환하는 일이다.
단순히 특정 단어가 등장했을 때 카운팅하게 만든다면 한국어의 특성상 상당히 많은 경우를 예외 처리해야한다.
(ex) 머리가 아파요, 머리가 뜨거워요, 짜끈거려요 -> 두통)

이를 해결하기 위해 문장 입력을 벡터로 치환하고 이를 db에 있는 단어 벡터들과 내적하는 과정을 통해 자연어로 입력된 증상에 대해 top-k 형식으로 문장에 들어간 증상이 무엇인지 영어로 나타낼 수 있었다.

데이터셋에 들어갈 칼럼들에 한에 결과를 제한하는 방법을 사용하였다.

```
1  from sentence_transformers import SentenceTransformer, util
2  import torch
3
4  # 모델 로드
5  encoder = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
6
7  # 칼럼 제약 걸기
8  symptoms = ["headache", "fever", "cough", "skin rash",
9              "vomiting", "fatigue", "chest pain", "dizziness"]
10
11 # 위에 칼럼 벡터화
12 symptom_emb = encoder.encode(symptoms, convert_to_tensor=True)
13
14 def find_symptoms(text, threshold=0.4, topk=5):
15     q = encoder.encode(text, convert_to_tensor=True)
16     scores = util.cos_sim(q, symptom_emb)[0] # 코사인 유사도
17
```

```
18     # 정렬
19     ranked = sorted(zip(symptoms, scores.tolist()), key=lambda x: -x[1])
20     print(f"\n입력: {text}")
21     for name, score in ranked[:topk]:
22         mark = "[v]" if score > threshold else "[-]"
23         print(f"    {mark} {name:12s} {score:.3f}")
24
25
26     find_symptoms(input())
27
```

데이터셋 가공

데이터셋 가공을 기획하였으나 24만 칼럼짜리 데이터셋을 찾았기에 우선 보류함

학습

전처리

- 1

2

3

4

5

6
1. int8로 읽기

2. diseases 컬럼 분리

3. 희귀 질병 제외

샘플 50개 미만 잘라내기

4. 죽은 증상 컬럼 제거

전부 0인 컬럼 정리

5. 라벨 인코딩

질병명을 숫자로

6. 층화 분할

비율 유지하며 train/test

```
epoch 4/30 | loss 0.5196 | 1.4s
epoch 5/30 | loss 0.4911 | 1.4s
epoch 6/30 | loss 0.4684 | 1.4s
epoch 7/30 | loss 0.4550 | 1.4s
epoch 8/30 | loss 0.4417 | 1.4s
epoch 9/30 | loss 0.4329 | 1.4s
epoch 10/30 | loss 0.4253 | 1.5s
epoch 11/30 | loss 0.4197 | 1.4s
epoch 12/30 | loss 0.4136 | 1.5s
epoch 13/30 | loss 0.4061 | 1.5s
epoch 14/30 | loss 0.4034 | 1.4s
epoch 15/30 | loss 0.3993 | 1.5s
epoch 16/30 | loss 0.3938 | 1.5s
epoch 17/30 | loss 0.3912 | 1.4s
epoch 18/30 | loss 0.3867 | 1.5s
epoch 19/30 | loss 0.3832 | 1.5s
epoch 20/30 | loss 0.3810 | 1.4s
epoch 21/30 | loss 0.3784 | 1.5s
epoch 22/30 | loss 0.3737 | 1.4s
epoch 23/30 | loss 0.3726 | 1.4s
epoch 24/30 | loss 0.3712 | 1.5s
epoch 25/30 | loss 0.3683 | 1.5s
epoch 26/30 | loss 0.3662 | 1.5s
epoch 27/30 | loss 0.3634 | 1.5s
epoch 28/30 | loss 0.3618 | 1.6s
epoch 29/30 | loss 0.3611 | 1.5s
epoch 30/30 | loss 0.3576 | 1.5s
```

```
===== 평 가 =====
top-1 정 확 도 : 0.8613
top-3 정 확 도 : 0.9611
top-5 정 확 도 : 0.9813
```

전체 파이프라인

한국어 문장 -> 임베딩 모델 -> 영어 칼럼명 -> 학습된 모델-> 질병 top-k 문제점

- 1

2

3

4

5

6

7

8

9

10

11

12

13

14

15
- >> 증상: 머리가 아프고 두통이 있고 속이 안 좋아요

입력: 머리가 아프고 두통이있고 속이 안좋아요

인식된 증상:

- dizziness (0.409)

- hoarse voice (0.43)

- sore throat (0.43)

- cough (0.443)

- feeling ill (0.449)

- vomiting (0.4)

- headache (0.566)

- nausea (0.409)

- facial pain (0.413)

- wheezing (0.558)

- ear pain (0.433)

- ringing in ear (0.458)

16	- plugged feeling in ear (0.424)	
17	- frontal headache (0.487)	
18	- feeling cold (0.444)	
19	- slurring words (0.481)	
20	- symptoms of the face (0.404)	
21	- paresthesia (0.413)	
22	- fever (0.429)	
23	- tongue lesions (0.436)	
24	- ache all over (0.424)	
25	- chills (0.5)	
26	- melena (0.558)	
27	- pulling at ears (0.453)	
28	- redness in ear (0.419)	
29	- nosebleed (0.459)	
30	- vulvar sore (0.429)	
31	- mouth pain (0.43)	
32	- sneezing (0.502)	
33	- hysterical behavior (0.403)	
34	- throat redness (0.438)	
35	- redness in or around nose (0.413)	
36	- sore in nose (0.436)	
37		
38	의심 질병 top-5:	
39	eustachian tube dysfunction (ear disorder)	60.8%
40	otitis externa (swimmer's ear)	38.9%
41	tinnitus of unknown cause	0.2%
42	sensorineural hearing loss	0.0%
43	meniere disease	0.0%

정확도가 60퍼센트 정도로 찍혔지만 실제 한국어로 대화한 것들 중 겹치지 않는 칼럼들도 높게 찍힘을 확인 할 수 있다. 그래서 이를 고치는 방안에 대해 탐색하였다.

개선

우선 이를 개선하기 위해 근본적이고 가장 처음인 임베딩 모델로 돌아가보았다.
데이터셋 칼럼이 영문명을 사용하기에 한국어 문장과 영어 단어를 내적하였는데 이에 대한 정확도는